```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score

pd.set_option("display.max_columns", None)
sns.set_theme(style="whitegrid")
```

```
df = pd.read_csv("iiot_network_data.csv")
df.head()
```

	timestamp	node_id	traffic_type	transmission_probability	capture_threshold	num_nodes	channel_quality	age_of_informa
0	2024-06-30 17:10:10.430548	61	deadline-oriented	0.9	-0.5	3	0.6	4.76
1	2024-07-01 03:12:10.430548	55	AoI-oriented	0.4	-2.0	2	0.7	4.06
2	2024-06-30 17:44:10.430548	63	deadline-oriented	0.3	0.0	4	0.6	19.00

```
print("Dataset shape:", df.shape)
print("\nColumn names:")
print(df.columns.tolist())

print("\nMissing values:")
print(df.isnull().sum())

print("\nData types:")
print(df.dtypes)

display(df.describe(include="all"))
```

```
Dataset shape: (10000, 9)

Column names:
['timestamp', 'node_id', 'traffic_type', 'transmission_probability', 'capture_threshold', 'num_nodes', 'channel_quality', 'age_o

Missing values:
timestamp                   0
node_id                     0
traffic_type                0
transmission_probability    0
capture_threshold           0
num_nodes                   0
channel_quality             0
age_of_information          0
packet_loss_probability     0
dtype: int64

Data types:
timestamp                    object
node_id                       int64
traffic_type                 object
transmission_probability    float64
capture_threshold           float64
num_nodes                     int64
channel_quality             float64
age_of_information          float64
packet_loss_probability     float64
dtype: object
```

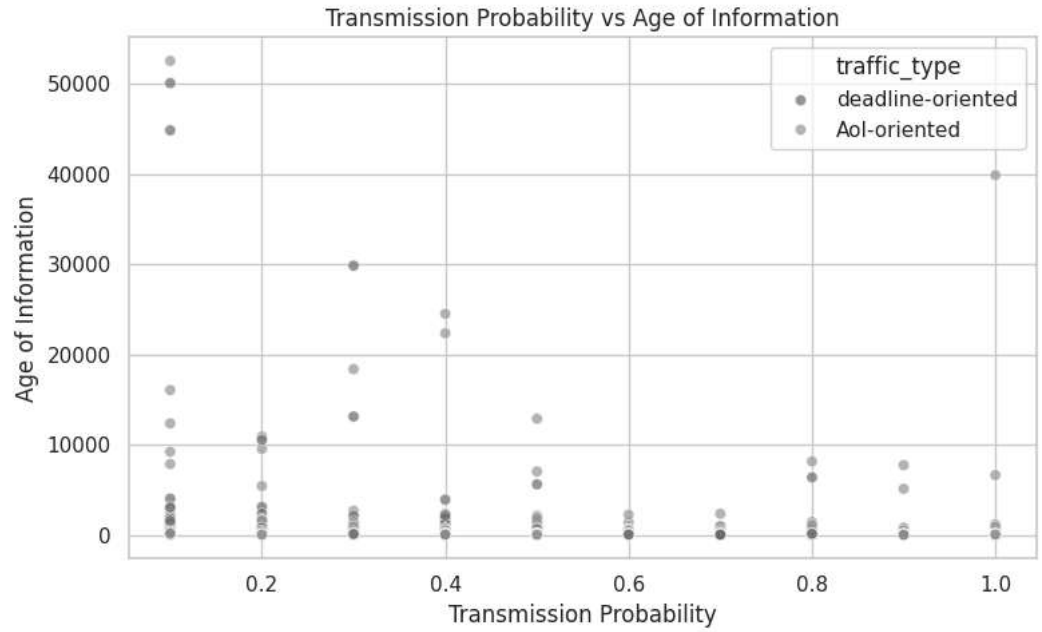
	timestamp	node_id	traffic_type	transmission_probability	capture_threshold	num_nodes	channel_quality	ag
count	10000	10000.000000	10000	10000.000000	10000.000000	10000.000000	10000.000000	
unique	1000	NaN	2	NaN	NaN	NaN	NaN	
top	2024-07-01 03:55:10.430548	NaN	deadline-oriented	NaN	NaN	NaN	NaN	
freq	20	NaN	5043	NaN	NaN	NaN	NaN	
mean	NaN	50.638400	NaN	0.548460	-0.001800	5.553100	0.499100	
std	NaN	29.020101	NaN	0.288548	1.284664	2.850122	0.317656	
min	NaN	1.000000	NaN	0.100000	-2.000000	1.000000	0.000000	
25%	NaN	26.000000	NaN	0.300000	-1.000000	3.000000	0.200000	
50%	NaN	51.000000	NaN	0.500000	0.000000	6.000000	0.500000	
75%	NaN	76.000000	NaN	0.800000	1.000000	8.000000	0.800000	
max	NaN	100.000000	NaN	1.000000	2.000000	10.000000	1.000000	

```
# Make a clean copy
df_clean = df.copy()

# Replace positive/negative infinity with NaN
df_clean = df_clean.replace([np.inf, -np.inf], np.nan)

# Check how many missing values we now have
print(df_clean.isnull().sum())

# Drop rows where age_of_information is missing
df_clean = df_clean.dropna(subset=["age_of_information"])

print("Cleaned dataset shape:", df_clean.shape)
display(df_clean.head())
```

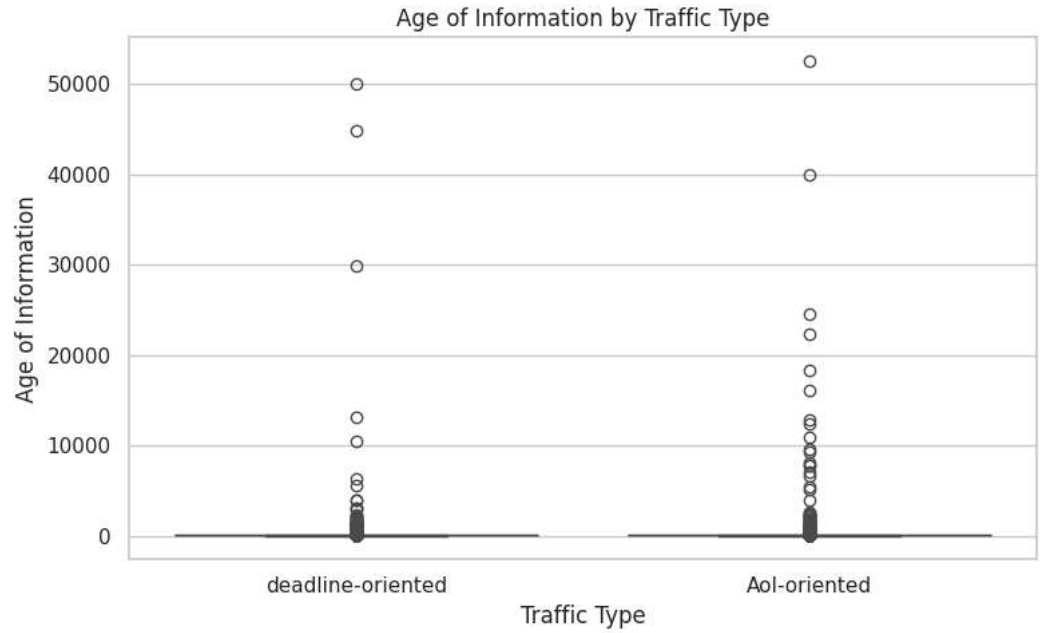
```
timestamp                    0
node_id                      0
traffic_type                 0
transmission_probability     0
capture_threshold            0
num_nodes                    0
channel_quality              0
age_of_information        1397
packet_loss_probability      0
dtype: int64
Cleaned dataset shape: (8603, 9)
```

	timestamp	node_id	traffic_type	transmission_probability	capture_threshold	num_nodes	channel_quality	age_of_informa
0	2024-06-30 17:10:10.430548	61	deadline-oriented	0.9	-0.5	3	0.6	4.76
1	2024-07-01 03:12:10.430548	55	AoI-oriented	0.4	-2.0	2	0.7	4.06
2	2024-06-30 17:44:10.430548	63	deadline-oriented	0.3	0.0	4	0.6	19.00

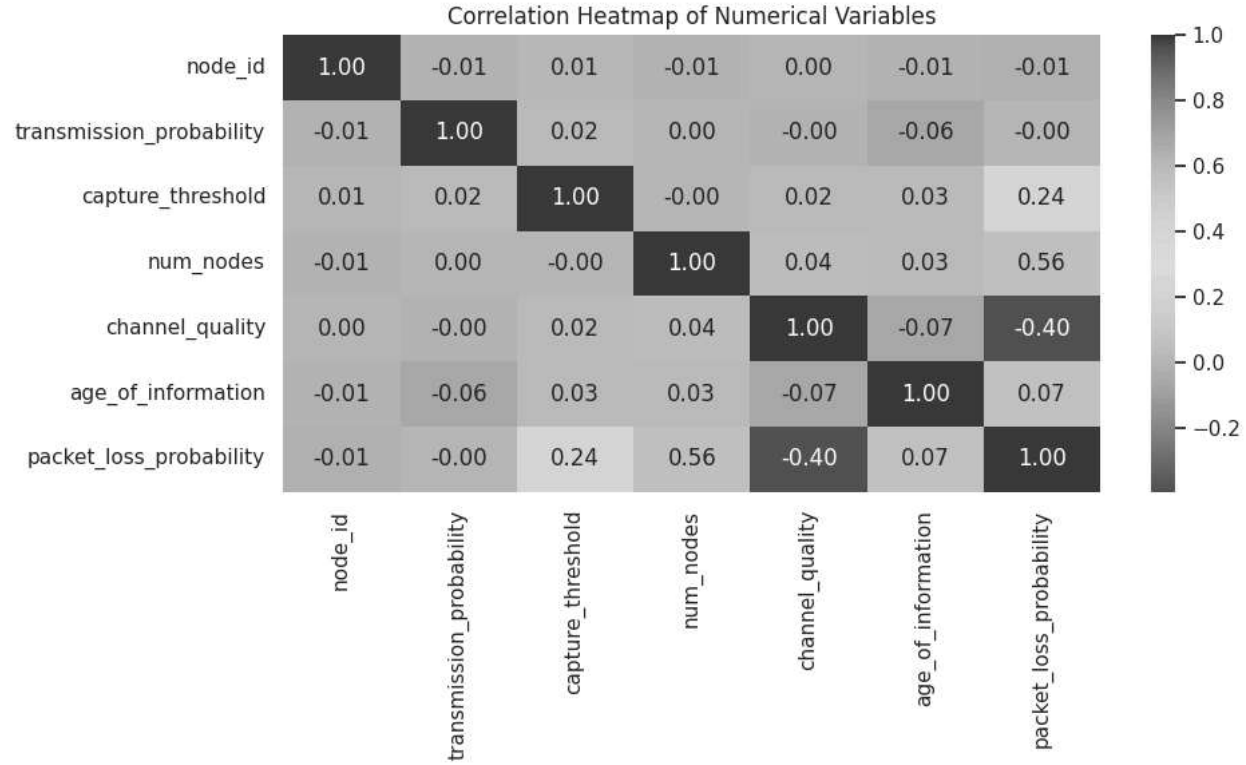
```
plt.figure(figsize=(8,5))
sns.scatterplot(
    data=df_clean,
    x="transmission_probability",
    y="age_of_information",
    hue="traffic_type",
    alpha=0.7
)
plt.title("Transmission Probability vs Age of Information")
plt.xlabel("Transmission Probability")
plt.ylabel("Age of Information")
plt.tight_layout()
plt.show()
```




```
plt.figure(figsize=(8,5))
sns.boxplot(
    data=df_clean,
    x="traffic_type",
    y="age_of_information"
)
plt.title("Age of Information by Traffic Type")
plt.xlabel("Traffic Type")
plt.ylabel("Age of Information")
plt.tight_layout()
plt.show()
```




```
numeric_df = df_clean.select_dtypes(include=[np.number])

plt.figure(figsize=(10,6))
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap of Numerical Variables")
plt.tight_layout()
plt.show()
```



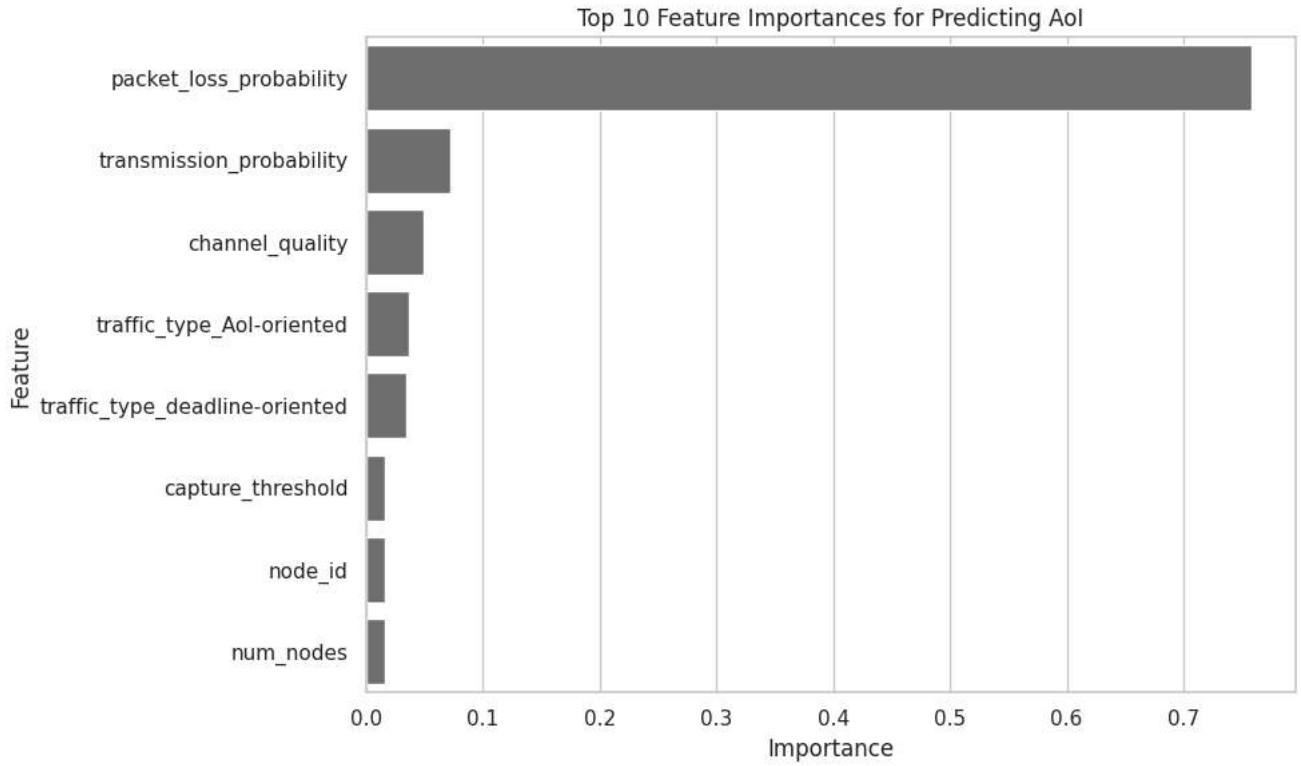

```
# Define target
target = "age_of_information"

# Features and target
X = df_clean.drop(columns=[target, "timestamp"])  # drop timestamp for now
y = df_clean[target]

# Identify numeric and categorical columns
numeric_features = X.select_dtypes(include=[np.number]).columns.tolist()
```

```
categorical_features = X.select_dtypes(exclude=[np.number]).columns.tolist()

print("Numeric features:", numeric_features)
print("Categorical features:", categorical_features)

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Preprocessing
preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_features)
    ]
)

# Random Forest pipeline
rf_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("model", RandomForestRegressor(n_estimators=200, random_state=42))
])

# Train model
rf_pipeline.fit(X_train, y_train)

# Predictions
y_pred = rf_pipeline.predict(X_test)

# Evaluation
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R-squared:", r2)
```

```
Numeric features: ['node_id', 'transmission_probability', 'capture_threshold', 'num_nodes', 'channel_quality', 'packet_loss_prob
Categorical features: ['traffic_type']
Mean Squared Error: 957742.8499403407
R-squared: 0.5650241549946695
```

```
ohe = rf_pipeline.named_steps["preprocessor"].named_transformers_["cat"]
encoded_cat_names = list(ohe.get_feature_names_out(categorical_features)) if len(categorical_features) > 0 else []

feature_names = numeric_features + encoded_cat_names
importances = rf_pipeline.named_steps["model"].feature_importances_

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importances
}).sort_values(by="Importance", ascending=False)

display(importance_df)

plt.figure(figsize=(10,6))
sns.barplot(data=importance_df.head(10), x="Importance", y="Feature")
plt.title("Top 10 Feature Importances for Predicting AoI")
plt.tight_layout()
plt.show()
```

	Feature	Importance
5	packet_loss_probability	0.757987
1	transmission_probability	0.071989
4	channel_quality	0.049045
6	traffic_type_AoI-oriented	0.037337
7	traffic_type_deadline-oriented	0.034773
2	capture_threshold	0.016767
0	node_id	0.016176
3	num_nodes	0.015926



Next steps: Generate code with `importance_df` New interactive sheet

```
new_configs = pd.DataFrame([
    {
        "node_id": 25,
        "traffic_type": "AoI-oriented",
        "transmission_probability": 0.2,
        "capture_threshold": -1.0,
        "num_nodes": 3,
        "channel_quality": 0.8,
        "packet_loss_probability": 0.20
    },
    {
        "node_id": 50,
        "traffic_type": "deadline-oriented",
        "transmission_probability": 0.5,
        "capture_threshold": 0.0,
        "num_nodes": 6,
        "channel_quality": 0.5,
        "packet_loss_probability": 0.60
    },
    {
        "node_id": 75,
        "traffic_type": "AoI-oriented",
        "transmission_probability": 0.9,
        "capture_threshold": 1.0,
        "num_nodes": 2,
        "channel_quality": 0.9,
        "packet_loss_probability": 0.10
```

```
    }
])

predicted_aoi = rf_pipeline.predict(new_configs)
new_configs["predicted_age_of_information"] = predicted_aoi
display(new_configs)
```

	node_id	traffic_type	transmission_probability	capture_threshold	num_nodes	channel_quality	packet_loss_probability	pr
0	25	AoI-oriented	0.2	-1.0	3	0.8	0.2	
1	50	deadline-oriented	0.5	0.0	6	0.5	0.6	

Next steps: Generate code with new_configs | New interactive sheet

Start coding or generate with AI.